

Identifying Erroneous Data in Image Datasets: A Comparison of Particle Swarm Optimization and Genetic Algorithms

Popescu Matei-Calin

June 1, 2026

Abstract

This paper compares two self-organizing optimizers, Particle Swarm Optimization and Genetic Algorithms on the task of identifying erroneous data in an image dataset. The detection problem is viewed as clustering in a PCA-reduced feature space, with the algorithms having to optimize a set of K centroids to minimize the sum of squared distances within each cluster. Four kinds of errors are injected into a subsampled set of Fashion-MNIST, with PSO and GA reaching competitive F1 scores of ≈ 0.327 against k-means at 0.356. The experiments show that errors within the data can be flagged reliably, while label errors require a different approach, and that overoptimizing the objective of lowering the SSE can negatively impact the F1 score.

1 Introduction

In real world situations, large datasets can be flawed. There are many situations where errors slip in datasets, whether that be due to mishandling (wrong labels, wrong data) or actual corruption of the data itself. Thus, an important task is sanitizing and checking for erroneous data, which becomes impractical to do manually on a large scale.

This paper aims to explore the effectiveness of two different self-organizing methods in flagging erroneous data in image datasets. I ran experiments at a small scale on the Fashion-MNIST dataset, comparing Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) with the more conventional k-means clustering. To introduce erroneous data, my experiments use four types of errors: occlusions, out of distribution data (regular MNIST data), random label flipping, and similar class swaps.

In order to achieve this, I used a geometric approach. In any feature space, similar samples will cluster together, with errors usually being geometrically further away. The task then becomes to identify the best possible set of points in the feature space around the clusters, which can be optimized in multiple ways, such as the methods explored in this paper, followed by flagging the data that is at a certain threshold away from each centroid as erroneous.

2 State of the Art

Particle Swarm Optimization (PSO), introduced by Kennedy and Eberhart [1], and Genetic Algorithms (GA), introduced by Holland [2], are two population-based self-organized search algorithms. PSO, as its name implies, works with a swarm of particles that search the feature space. These particles are influenced by two weights, the best position found, and the best

position found by the swarm. GA uses a population that optimizes generation after generation through mutation, crossover, elitism, etc.

In these experiments, clustering is used as an optimization problem, i.e. driving down the sum of squared distances (SSE) within each cluster. In [3], van der Merwe and Engelbrecht, as an alternative to k-means, proposed using particles as a set of user defined centroids, guided by PSO to minimize the SSE, reporting similar performance to k-means and their results showing PSO clustering has much potential.

Along with PSO, Genetic Algorithms can be used for the same task of minimizing the within cluster SSE. these two approaches share the initial encoding, that being a particle or chromosome representing the entire set of centroids. The distance from a data point to the nearest centroid determines how well that point fits within the cluster, with points further away from a centroid being possible outliers or erroneous. Such distance-based scoring is mentioned as an important method of anomaly detection by Chandola [4].

Unlike the mentioned works, my experiments compare PSO and GA directly on a fixed dataset (Fashion-MNIST), judging detection quality (F1 score) against the SSE minimized by the algorithms, with k-means as a baseline.

3 Method

3.1 Dataset and Error Injection

Given performance restraints, the dataset used in these experiments is Fashion-MNIST, 28x28 images of clothing items with 10 classes. This dataset was further subsampled from 70000 to 10000 data points.

To introduce errors, I used 4 different types:

- Occlusion: a 10x10 block of zeroed pixels
- Out of Distribution: images from a completely different dataset (MNIST handwritten numbers) replacing original data, keeping the same label
- Random label flipping
- Class swap on similar pairs: T-Shirts with Shirts, Pullovers with Coats, Sandals with Sneakers

Each error type replaces 4% of the data, which gives a total of 16% of the subsampled dataset being erroneous.

Flattening the 28x28 images results in a 784-dimensional feature space. At a number of K centroids, a candidate solution would search a very high dimensional space ($K \times 784$), and due to the curse of dimensionality, which makes Euclidean distances almost uniform with higher dimensions, Principal Component Analysis (PCA) was applied to reduce the feature space while preserving the axes with the most variance in the different classes.

In Figure 1, variance is plotted against the number of PCA components (dimensions). The curve climbs nearly vertically through the first ~ 40 dimensions, then flattens. I have found 50 to be the best compromise between dimensions and variance, capturing $\sim 82\%$ of the variance.

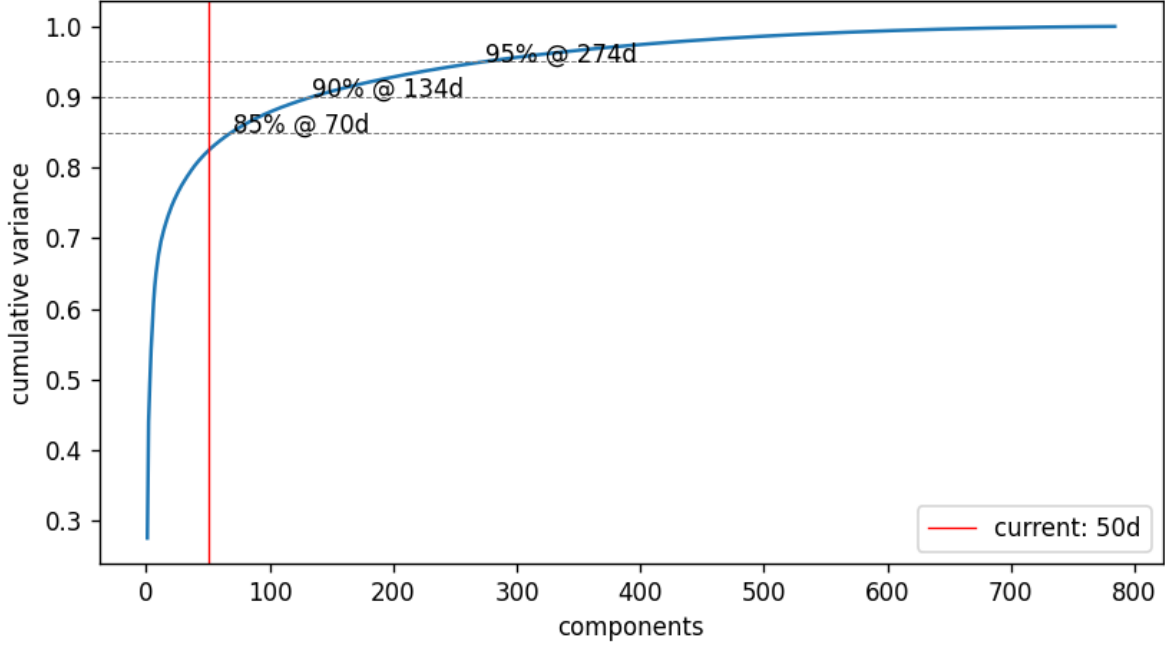


Figure 1: PCA tradeoff between dimensions and cumulative variance

Chasing more gives diminishing returns, with 85% requiring 70 dimensions, 90% requiring 134 and 95% requiring 274.

3.2 Optimizers

All three methods solve the same problem: find K centroids such that the within-cluster sum of squared distances:

$$\text{SSE} = \sum_i \min_k \|x_i - c_k\|^2,$$

is as small as possible. A solution is the entire set of centroids of K 50-D points, with the encoding and the fitness being shared across PSO, GA, and k-means. This ensures an identical setup for a fair comparison between the three methods.

In the experiments, as a middle ground between number of centroids, performance and clustering capability, for both algorithms, I have settled on $K = 20$ centroids, swarm/population size of 50 and 1000 iterations/generations. The set of centroids for each particle/chromosome at the beginning of a run are initialized from K random points in the data.

PSO uses a swarm of such candidate solutions, with each particle remembering the best position it has visited (p_{best}) and also being pulled toward the best the entire swarm has found (g_{best}), updating its velocity and position at each iteration:

$$v \leftarrow wv + c_1r_1(p_{\text{best}} - x) + c_2r_2(g_{\text{best}} - x), \quad x \leftarrow x + v,$$

with r_1, r_2 within $[0, 1]$ and $c_1 = c_2 = 1.5$. The inertia w drops linearly from 0.9 to 0.4 over the experiments, allowing the swarm to search more widely earlier on, followed by slowing down later for refinement.

GA uses the same initial encoding, with the difference being it evolves through selection, crossover, and mutation. Parents are chosen by tournament selection of size 3, crossover is applied per centroid (they are supposed to be copied whole from the parents, I ran into the issue of mixed coordinates that produced bad data and cost performance), mutation then adds

Gaussian noise ($\sigma = 0.5$) to each coordinate with a probability 0.02, and elitism lets the best 4 chromosomes remain intact the next generation.

k-means is the reference, simple baseline, I used to compare against the two self-organizing algorithms. The standard scikit-learn implementation was used with the same $K = 20$ centroids on the same data, with the same objective of minimizing the SSE.

3.3 Scoring and evaluation

Once an algorithm has found a set of optimum centroids, each point in the dataset gets an anomaly score based on the Euclidean distance to the nearest centroid: $s(x) = \min_k \|x - c_k\|_2$. Points further from every centroid score higher, and a set fraction of those gets flagged. Since in these experiments I know the injected errors make up exactly 16% of the data, I flag the top 16%, placing the cutoff at the 0.84 quantile of the scores.

4 Results

4.1 Performance

Looking at the results shown in Table 1, k-means performs the best, with the highest F1 score (0.356). PSO and GA are, however, competitive, with F1 scores of ≈ 0.327 . k-means does the best job at minimizing the SSE, achieving a mean SSE of 18.7. PSO and GA are further behind, at 26.75 and 25.74 respectively. The three methods rank the same in AUC score too, with k-means leading at 0.66 and PSO and GA closely behind at ≈ 0.62 .

Method	F1	AUC	SSE/ N
k-means	0.356	0.66	18.7
PSO	0.328	0.62	26.75
GA	0.326	0.61	25.74

Table 1: Detection performance at the 16% cutoff. F1 = precision = recall by construction.

Also, as a consequence for the experiments setup, by having a fixed cutoff of 16% and a true error rate of 16%, precision and recall become the same, and thus the F1 score is equal to both of them too: $F1 = \text{precision} = \text{recall}$.

4.2 Convergence

The paths that the two self-organizing algorithms take to converge at a lower SSE differ, as shown in Figure 2. PSO uses most of the iterations to explore the feature space, with the SSE maintaining a high value until iteration ≈ 750 , when it starts to refine and the SSE drops sharply until iteration 1000, at a final mean SSE of 26.75. GA does the reverse, with a sharp drop in SSE in the first ≈ 100 generations, followed by a steady decline until generation 1000 through a combination of elitism and low mutation rate, finally arriving at a mean SSE of 25.74.

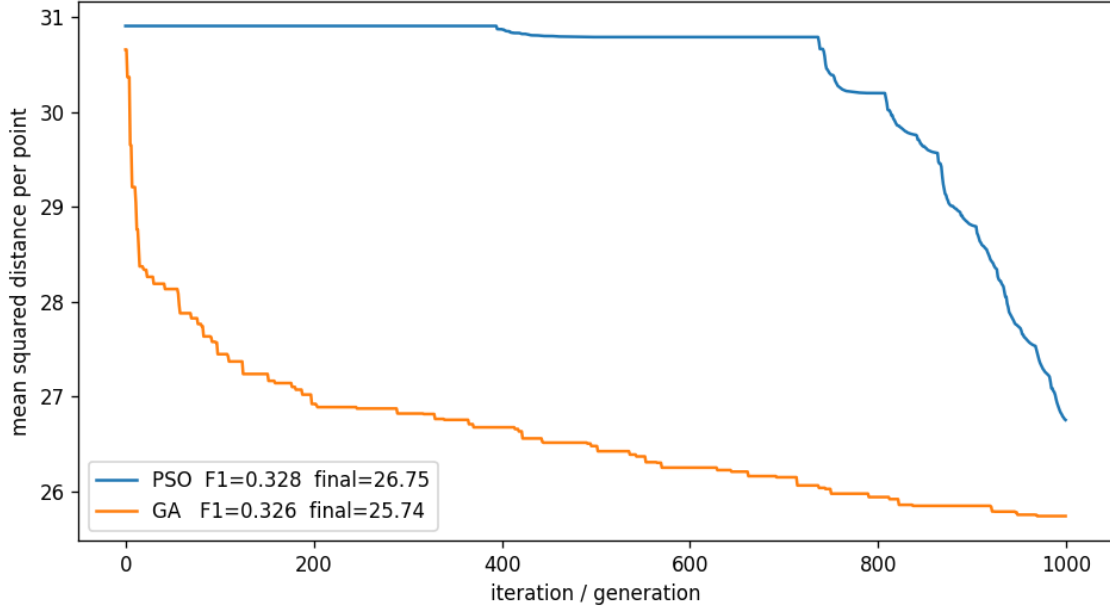


Figure 2: Mean SSE per iteration/generation of PSO/GA, k-means as reference

Perhaps the most interesting finding is that the F1 scores do not always correlate with the SSE that the optimizers are trying to minimize, as seen in Figure 3. PSO begins with its highest F1 score of ≈ 0.34 , which remains stable through most of its initial plateau in SSE. Only towards the end, once PSO starts refining and the SSE drops, the F1 score starts falling, fluctuating until it reaches its final value of 0.328. GA starts with a much lower F1 score, which sees a sharp increase in the first ≈ 100 generations, correlating with the initial drop in SSE. However, after the initial peak, it sees a drop back below 0.30, followed by a steady increase until its final value of 0.326.

Thus, the F1 scores and SSE are correlated, but do not always align, and in the case of PSO, optimizing SSE can even lead to a lower F1 score.

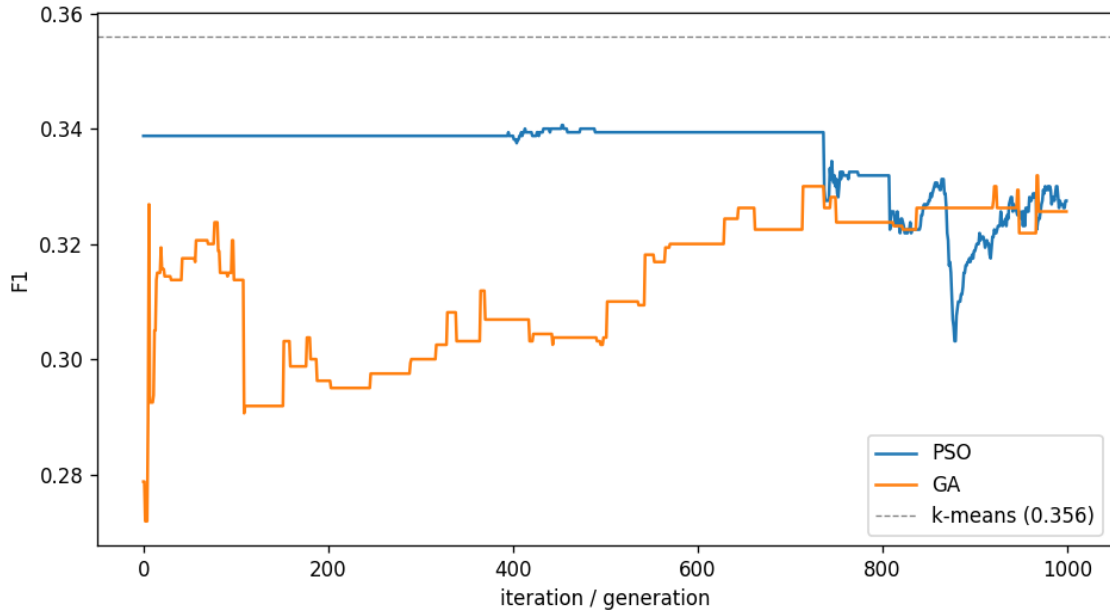


Figure 3: F1 trajectory of PSO and GA, k-means F1 as a reference

4.3 Performance by error types

Since the injected errors are of 4 different types, the two self-organizing algorithms perform differently on each. Because the nature of the experiments is geometric, the algorithms can only flag possibly erroneous data if it is geometrically far from the centroids. If the type of error modifies the contents of the image itself, such as the occlusion and out of distribution error types, all the algorithms do a good job at flagging them, as seen in the high recall values in Figure 4.

However, when the error is only at the level of the label, nothing in the actual image is modified, thus this kind of flawed data stays close to the centroids and the algorithms become blind to them, as seen with the sharp drop in recall for random label flipping and similar class swaps. Interestingly, the similar class swaps perform even worse than the random baseline of 0.16. This could be due to the points ending up closer to the centroids of the class they were swapped with, thus being even less likely to be flagged.

This is better visualized in Figure 5, where every algorithm is measured against every error type. The plots show the anomaly score (Euclidean distance to the nearest centroid) against the density of the data points, with the dashed line being the 16% cutoff. The algorithms clearly distinguish occlusion and OOD errors, with the distributions shifting further right, while the label error distributions are near identical to the clean data.

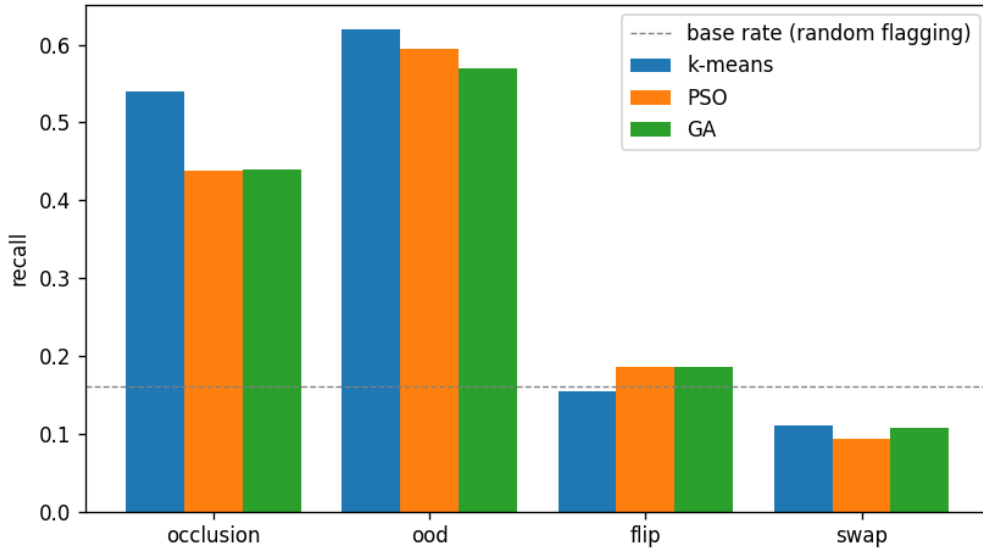


Figure 4: Recall per error type

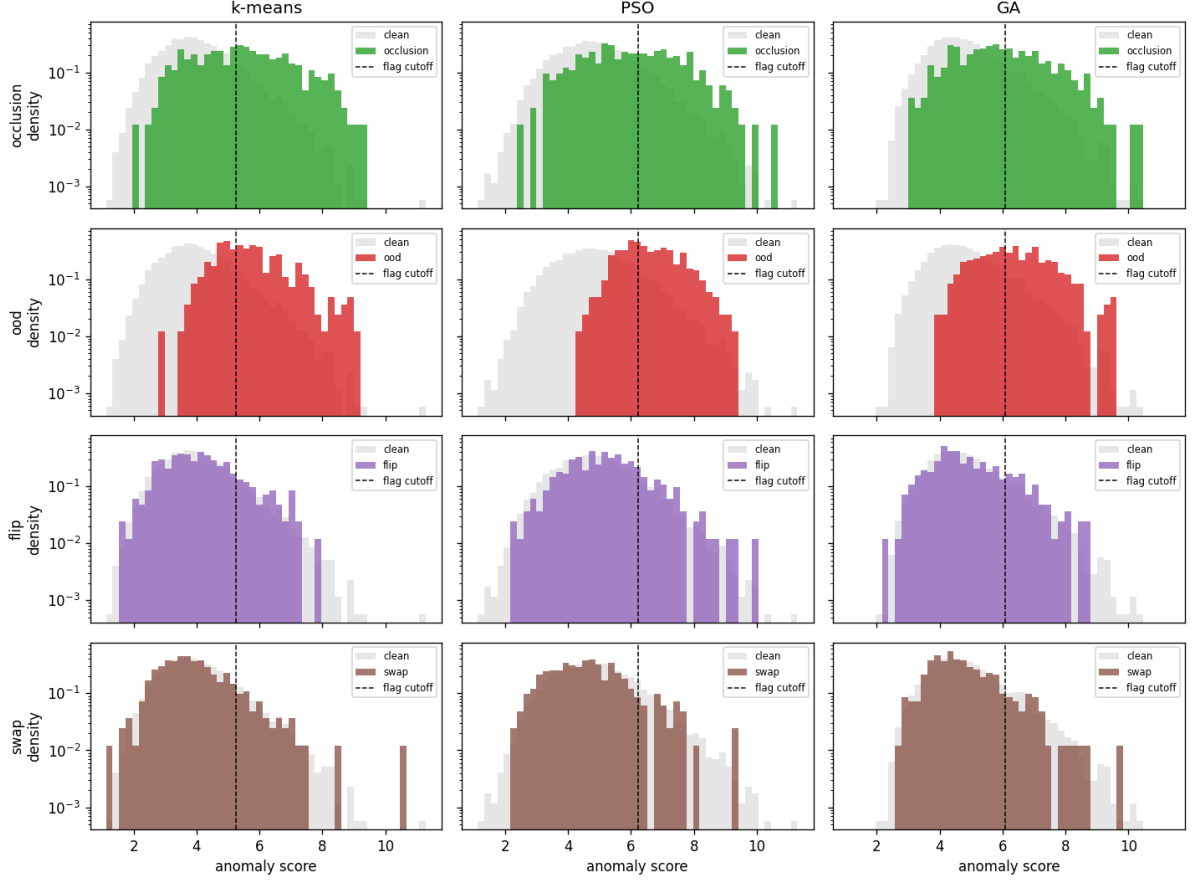


Figure 5: Anomaly score distributions by error kind

5 Conclusions

The small-scale experiments on detecting injected erroneous data in the Fashion-MNIST dataset show that self-organizing methods such as Particle Swarm Optimization and Genetic Algorithms are competitive with the widely used k-means clustering. Despite these two algorithms not overtaking k-means in this configuration, the subtle, interesting finding was that the SSE and F1 scores are correlated, but do not evolve exactly the same, with the overoptimization of one being able to negatively impact the other.

The biggest limitation of these methods is the kinds of errors they can flag, in this case, half of the errors (label errors) being functionally invisible. The algorithms can only cluster based on the actual content of the images, thus only the errors that modify the images can be flagged (occlusion and OOD). Another limitation of the experiment was the small scale, the single dataset and the fact that the true size of the error pool is known, which is not the case in real situations. To combat these limitations, future work could explore more complex algorithms, a smarter approach on building the feature space (e.g. a CNN classifier to flag label errors in conjunction with self-organizing methods later on).

References

- [1] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of ICNN’95-international conference on neural networks*, vol. 4, pp. 1942–1948, iee, 1995.

- [2] J. H. Holland, *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.
- [3] D. W. van der Merwe and A. P. Engelbrecht, “Data clustering using particle swarm optimization,” in *The 2003 Congress on Evolutionary Computation, 2003. CEC’03.*, vol. 1, pp. 215–220, IEEE, 2003.
- [4] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.